CS101: Data Structures and Algorithms

Unit 1: Introduction to Algorithm Complexity & Big-O

Computational complexity theory classifies algorithms according to how much time or memory they consume as input size N grows toward infinity. We focus on two primary resources: Time Complexity (the number of elementary operations performed) and Space Complexity (the peak auxiliary memory allocated during execution).

Asymptotic Notations & Order of Growth:

- **Big-O (O(N))**: Represents an asymptotic upper bound. For example, linear search is O(N).
- **Big-Omega (Ω(N))**: Represents an asymptotic lower bound of the algorithm's runtime.
- **Big-Theta (Θ(N))**: Represents a tight asymptotic bound where upper and lower bounds coincide.
- **O(1) Constant Time**: Hash table lookups on average, array indexing, and basic arithmetic.
- **O(log N) Logarithmic Time**: Binary search over a sorted array or balanced binary search tree operations.
- **O(N log N) Linearithmic Time**: Optimal comparison-based sorting algorithms like Merge Sort and Heap Sort.
- **O(N²) Quadratic Time**: Brute-force nested loops, Bubble Sort, Insertion Sort worst case.
- **O(2^N) Exponential Time**: Recursive Fibonacci without memoization, generating all subsets of a set.

Amortized Analysis: Dynamic arrays (e.g., Python lists or C++ std::vector) double their capacity whenever they fill up. While a resize operation takes O(N) copying time, it occurs infrequently enough that the average amortized cost per append operation remains strictly **O(1)**.

Unit 2: Linear Data Structures & Memory Layouts

Linear data structures arrange elements in sequential order. Choosing between arrays and linked representations depends fundamentally on memory layout and access patterns.

Arrays vs. Linked Lists Comparison:

Operation	Static Array	Singly Linked List	Doubly Linked List
Random Access (index i)	O(1)	O(N)	O(N)
Insert / Delete at Head	O(N)	O(1)	O(1)
Insert / Delete at Tail	O(1) amortized	O(N) (or O(1) with tail)	O(1) with tail pointer
Cache Locality	Excellent (Contiguous)	Poor (Scattered heap)	Poor (Scattered heap)
Per-element Memory Overhead	Zero extra pointers	1 pointer (8 bytes on x64)	2 pointers (16 bytes on x64)

Stacks and Queues:

• **Stack (LIFO - Last In, First Out)**: Fundamental operations are push() and pop(), both operating in O(1) time. Applications include the call stack execution frame in programming languages, undo mechanisms in text editors, and syntax parenthesis matching algorithms.
• **Queue (FIFO - First In, First Out)**: Operations include enqueue() and dequeue(). Implemented using circular ring buffers or linked lists. Applications include operating system CPU task scheduling, print spoolers, and Breadth-First Search (BFS) frontier queues.

Unit 3: Trees, Balanced Trees, and Graph Traversal

Hierarchical non-linear data structures allow efficient searching, sorting, and relationship modeling.

Binary Search Trees (BST) & Self-Balancing Trees:

In a standard BST, for any node N, all keys in N's left subtree are strictly less than N.key, and all keys in the right subtree are strictly greater. In-order traversal visits nodes in ascending sorted order.

• **Worst Case Degeneracy**: If elements are inserted in already-sorted order, a standard BST degenerates into a singly linked list with **O(N) search time**.

• **AVL Trees**: An AVL tree enforces a strict balance factor for every node: Height(LeftSubtree) - Height(RightSubtree) $\in$ {-1, 0, 1}. Whenever an insertion or deletion violates this invariant, tree balance is restored via O(1) tree rotations (Left rotation, Right rotation, Left-Right rotation, Right-Left rotation). Consequently, search, insertion, and deletion strictly guarantee **O(log N) worst-case time**.

Graph Algorithms & Pathfinding:

• **Breadth-First Search (BFS)**: Explores level by level using a FIFO queue. Time complexity is O(V + E). BFS is guaranteed to discover the shortest path in unweighted graphs.

• **Depth-First Search (DFS)**: Explores as deep as possible along each branch before backtracking using recursion or an explicit stack. Used for topological sorting and cycle detection in directed graphs.

• **Dijkstra's Algorithm**: Solves the single-source shortest path problem for graphs with non-negative edge weights. Using a Min-Heap priority queue, Dijkstra's algorithm achieves a time complexity of **O((V + E) log V)**.

CS101: Course Syllabus & Administrative Policies

Course Instructor: Professor Alan Turing | Office: Turing Hall 402
Teaching Assistant: Grace Hopper | Office Hours: Wednesdays 3:00 PM - 5:00 PM
Lecture Schedule: Mondays and Wednesdays 10:00 AM - 11:30 AM in Auditorium B

Grading Breakdown:

Component	Weight	Description
Programming Projects (4)	40%	Hands-on implementation of BST, Graph algorithms, and Caching
Midterm Examination	20%	Held on Week 7 covering Units 1 and 2 (Linear structures & Big-O)
Final Comprehensive Exam	30%	Scheduled during University Finals Week, covers all 4 units
Weekly Quizzes & Labs	10%	Online multiple-choice quizzes and laboratory participation

Assignment Late Policy:

All programming projects are due at 11:59 PM on the specified due date. Late submissions are penalized at a rate of **10% per calendar day** for up to 3 days (72 hours). Submissions submitted more than 72 hours late will automatically receive a **grade of 0%**, unless an official university medical extension is granted in advance.

Academic Integrity Code:

All submitted code must be your own individual work. Copying code from classmates, online repositories, or unauthorized generative tools without attribution is considered a direct violation of the honor code. First offenses result in a zero on the assignment and a formal letter to the Dean of Academic Affairs.